HOJA Nº 1

FECHA

Orientación a objetos 2. final

Patrones de diseño ①

Los patrones de diseño son soluciones habituales a problemas comunes. Usados p/ tener diseños + flexibles, extensibles y reusables. ya q' describen un problema y su solx p/ q' se reutilice muchas veces.

Cada patrón se compone por:
- Nombre: en una o dos palabras describe el problema y solx
- Descripx: cuándo aplicarlo y qué hace.
- Solx: elementos q' lo componen
- Consecuencias de los resultados.

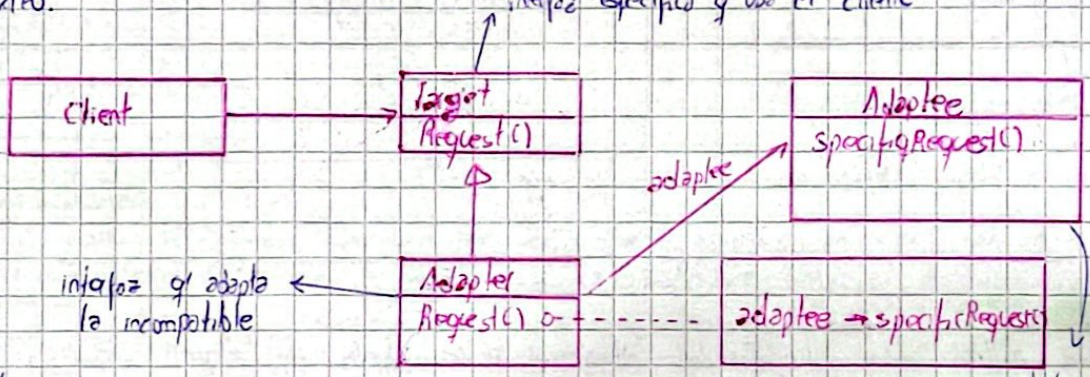
Patrones estructurales

Explican cómo ensamblar objetos y clases en estructuras más grandes a la vez q' se mantiene la flexibilidad y eficiencia.
Se basan en la composix y herencia. ya q' busca q' <> estructuras trabajen juntas o q' una clase simple maneje > complejidad
- Herencia: "es un", una clase adquiere propiedades y comportamientos de otra
- Composix: "tiene un", una clase contiene una referencia a un objeto de otra clase p/ delegar tareas.

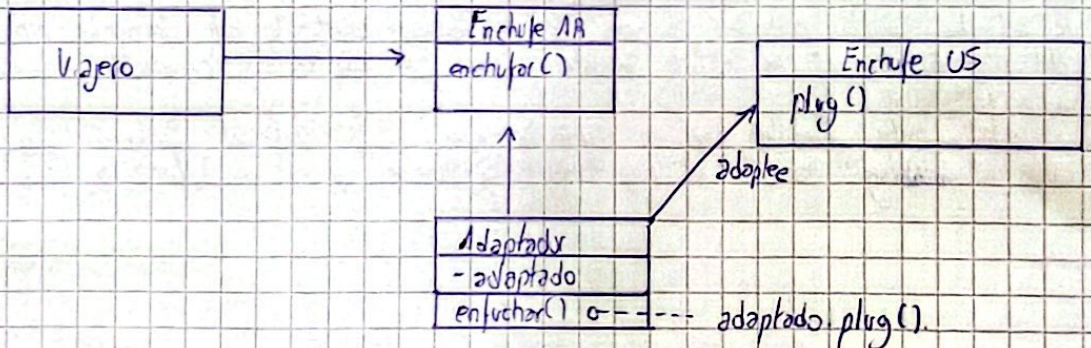
≠ wrappers, término informal p/ patrones q' envuelven, son: adapter, decorator, proxy.

Adapter pattern

Utilizado cuando es necesario que 2 clases incompatibles lo sean p/ q' trabajen en conjunto.



ejemplo

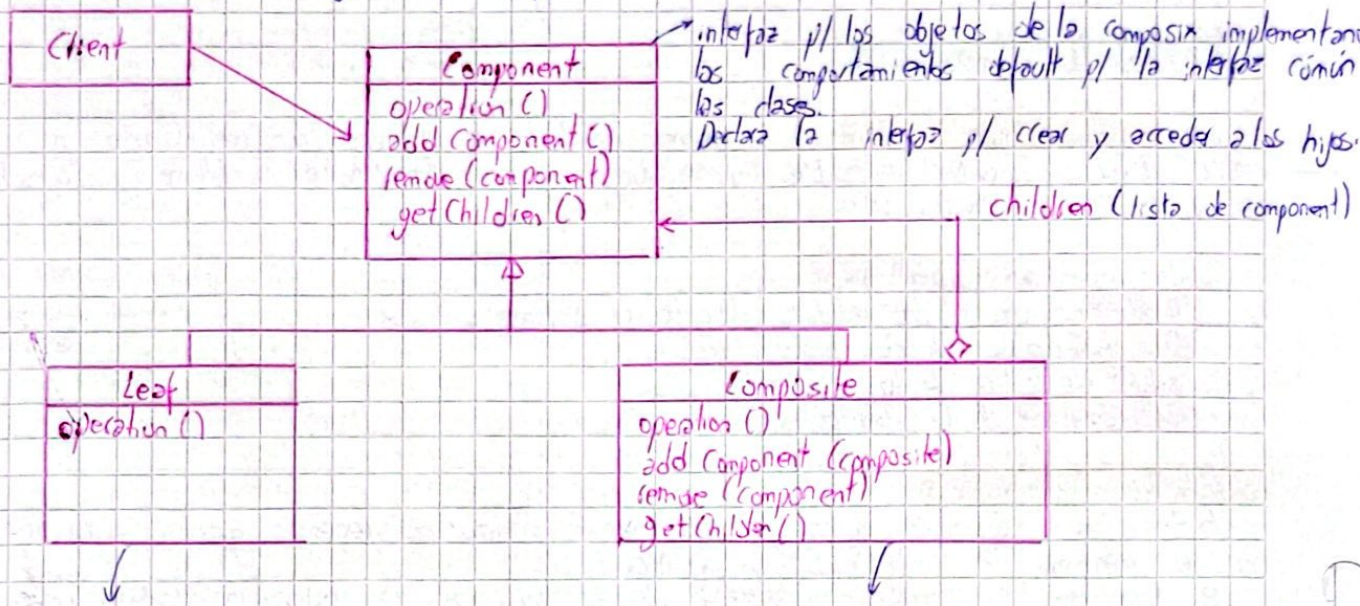


NOTA

Cliente: código q' usa el patrón

Composite pattern

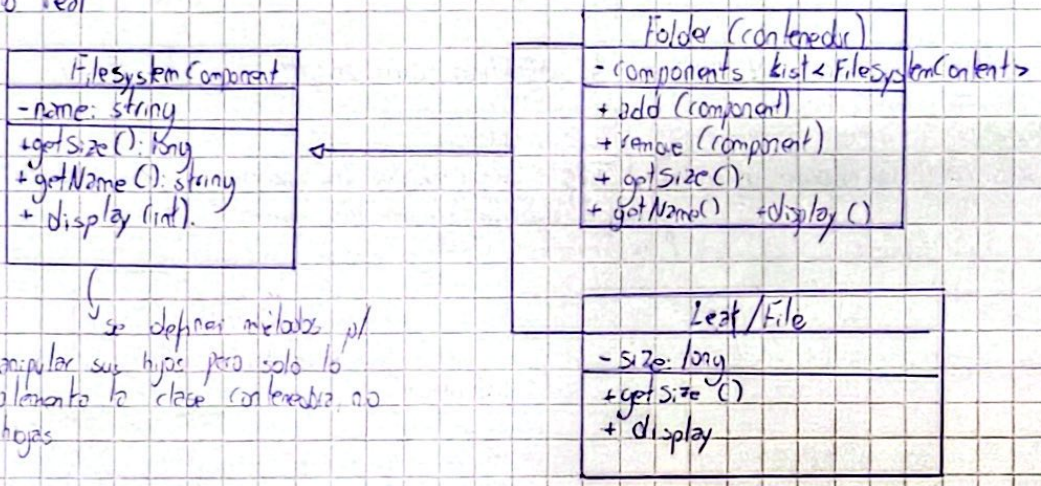
Usado para representar jerarquías en forma de árbol.



Representa los hojas y el comportamiento del primitivo es definido

Define el comportamiento p/ componentes c/ hijos, junto c/ su referencia además de operaciones.

ejemplo real



se definen métodos p/ manipular sus hijos pero solo lo implementa la clase contenedora no hojas

Decorator pattern

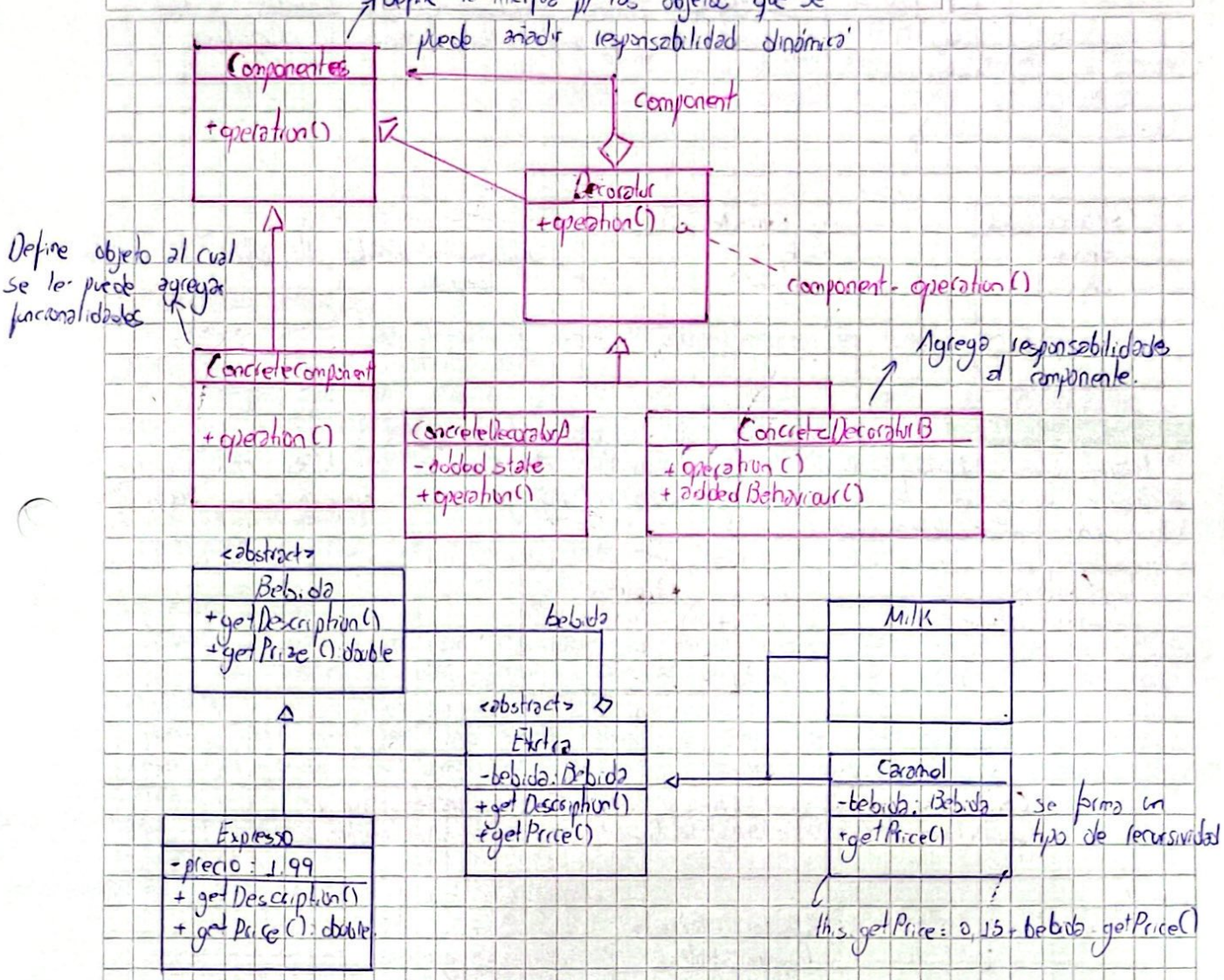
El decorator pattern agrega funcionalidad dinámica a un objeto de manera individual y transparente. Brinda flexibilidad ya que es mejor q' herencia. y es útil cuando subclasificar es impráctico.

Decorator vs adapter: ambos patrones decoran el objeto p/ cambiarlo, el decorator preserva la interface del objeto para el cliente mientras q' el adapter lo convierte en cambio el decorator añade funcionalidad

Decorator vs. strategy: ambos permiten q' un objeto cambie su funcionalidad dinámica. La diferencia es q' el strategy cambia el algoritmo x dentro y el decorator lo hace x fuera.

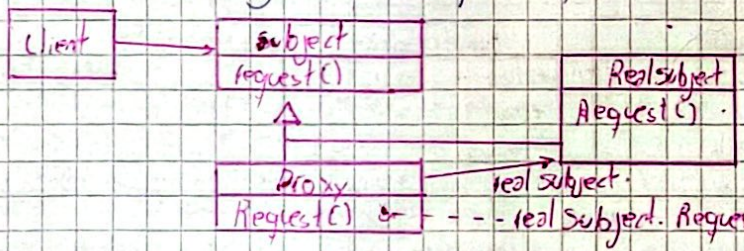
NOTA

Scanned with CamScanner



Proxy pattern

Actúa como intermediario/sustituto de otro objeto. Controla el acceso al objeto original permitiendo hacer algo antes o después q' la solicitud llegue al objeto original.



Patrones de comportamiento

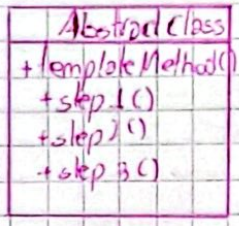
Tratan c/ algoritmos y la asignax de responsabilidades el objetos. p/ q' haya menor acoplamiento.

Template method

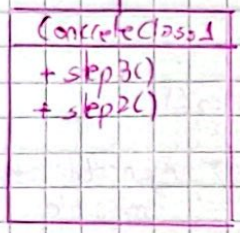
Define el esqueleto de un algoritmo en la super clase y permite q' las subclases sobreescriban pasos del algoritmo sin cambiar su estructura.

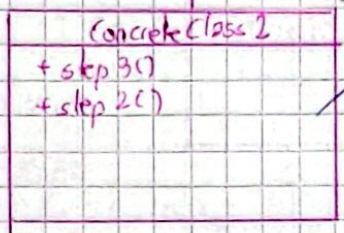
Es utilizado p/ implementar las partes invariables de un algoritmo una vez y en las subclases implementar lo que sí varía.

Scanned with CamScanner



→ define clases primitivas q' son definidas x las subclases p/ implementar pasos del algoritmo

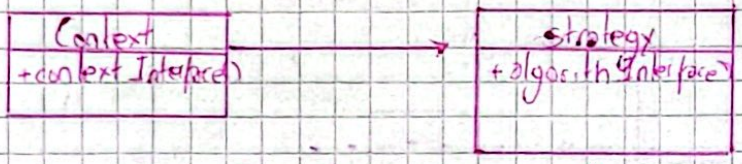


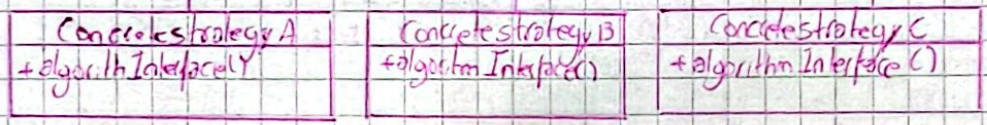


→ implementa métodos del padre haciendole overrides.

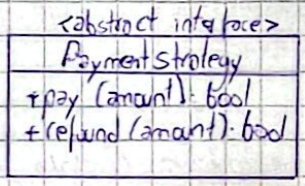
Strategy pattern: Desacopla algoritmos del objeto. Cambiado mientras

!! Tiene como objetivo desacoplar un algoritmo del objeto q' lo utiliza cambiándolos dinámico' en ejecuc. lo cual brinda flexibilidad p/ agregar nuevos algoritmos q' lleven a cabo una función determinada.

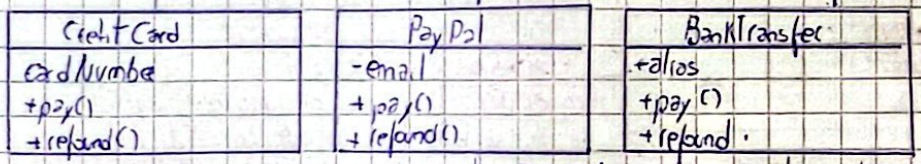




ejemplo



todos los medios tienen <> tipos de implementar p/ el mismo procedimiento (pay/refund)



State pattern: estado → comportamiento. El cambio de comportamiento es consecuente del cambio de estado.

El state pattern permite a un objeto alterar su comportamiento cuando su estado interno cambia.

Desacopla el estado interno del objeto en una jerarquía de clases donde cada una representa un estado concreto en el q' puede estar el objeto.

!! Cada cambio de estado lo hace el objeto y NO el cliente como en strategy.!!

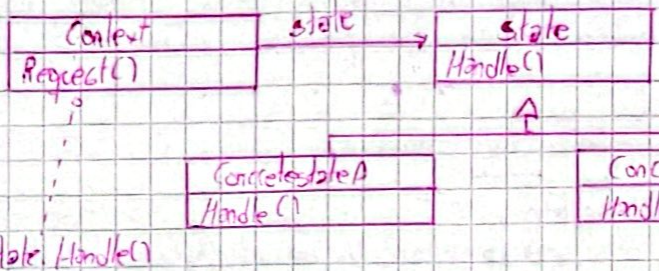
strategy = lo hace el mismo cliente

state: lo hace el objeto mismo.

NOTA

representar a crear.

Scanned with CamScanner



El contexto puede pasarse a sí mismo llama a Handle() y Handle() cambia el estado del contexto (instancia otro estado)

state.Handle()

State vs. strategy

Los cambios de state lo hace el estado mismo en cambio en strategy el cambio lo hace el contexto/cliente.

Strategy útil p/ permitir q' una clase delegue la ejecux de un algoritmo o una familia de estrategias

State útil p/ una clase q' tiene q' hacer entre estados fácil.

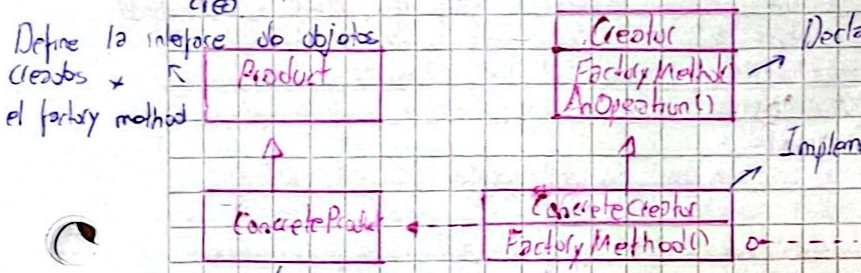
Patrones creacionales

Patrones ocupados de cómo se crean los objetos.

Factory method

Permite crear objetos sin especificar sus clases exactas. Define una interface p/ la creax de objetos, mientras permite que las subclases decidan qué clase se debe instanciar.

Útil p/ una clase que no puede prever la clase de objetos q' debe crear, una clase que quiere q' sean sus subclases quienes especifiquen los objetos que éste crea.

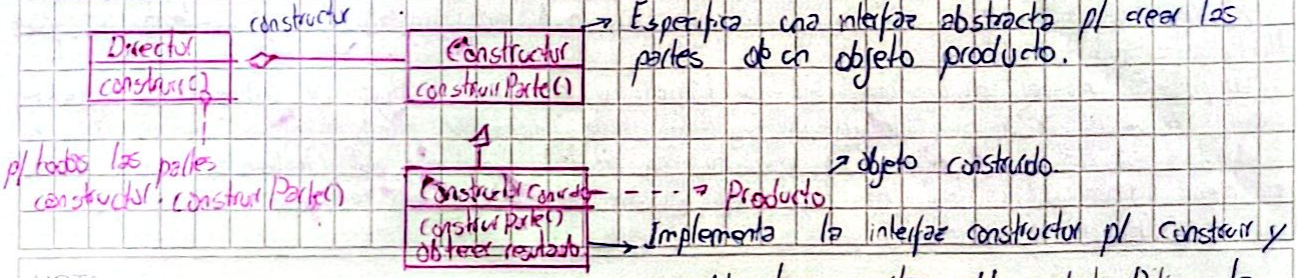


Builder

Utilizado p/ separar la construx de un objeto complejo de su representax (implementax) de tal manera q' el mismo proceso puede construir diferentes representaciones.

Usando El algoritmo p/ crear su objeto complejo debiera ser independiente de las partes de q' se compone dicho objeto y de cómo se ensamblan.

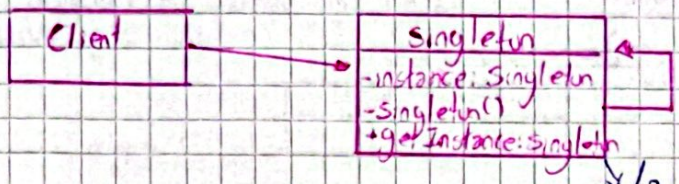
El proceso de construx debe permitir <> representaciones del objeto q' está siendo construido.



NOTA

Singleton pattern

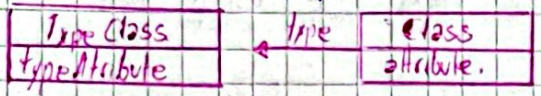
Garantiza que una clase tenga una única instancia cuando se necesita un único objeto compartido.
Una única instancia debería ser extensible mediante herencia y los clientes deberían ser capaces de usar una instancia extendida sin modificar su código.



La clase declara el método estático obtenerInstancia q' devuelve la misma instancia de su propia clase.

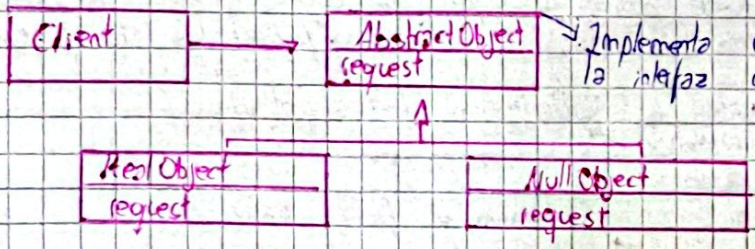
Type object pattern

Permite la creación flexible de nuevas clases creando una única clase donde cada instancia la cual representa diferentes tipos de objetos



Null object

Proporciona un sustituto p/ otro objeto q' comparte la misma interfaz pero no hace nada. El null object encapsula las decisiones de implementación de cómo "no hacer nada" y oculta esos detalles de sus colaboradores
Algunas instancias colaboradoras no deberían hacer nada.



Implementa el comportamiento predeterminado p/ la interfaz común a todas las clases s/ corresponda

(2) Refactoring

Reestructuración de código c/ el objetivo de mejorar la calidad del código y diseño sin cambiar su comportamiento. Se mejora p/ q' sea más fácil de entender y barato de modificar.

Leyes de Lehman

① Continuing change: sistemas deben adaptarse continuo' o se vuelven menos satisfactorios
② Continuing growth: la funcionalidad de un sistema debe incrementarse continuo' p/ la satisfac del cliente.
③ Increasing complexity: a medida de que un sistema evoluciona su complejidad, el trabajo tmb.
④ Declining quality: la calidad de un sistema va a ir declinando a menos de que haya un mantenimiento riguroso.

El refactoring se aplica p/ mejorar la organización, legibilidad, adaptabilidad, y mantenibilidad pero NO altera comportamiento.
Implicará eliminar duplicaciones, simplificar lógica y clarificar código. Y aplicaré mini refactoring cuando encuentro código complejo o antes de aplicar un cambio.
Ayudará a la deuda técnica la cual es un costo implícito del trabajo adicional causado x elegir una solución fácil en lugar de utilizar un enfoque q' llevaría más tiempo en su desarrollo e implementación

NOTA

Es importante el refactoring xq: gano comprens de código, reduzco costo de mantenimiento debido a los cambios inevitables que sufrirá el sistema como facilitar la detec de bugs.

CLEAN code: cohesivo, débilmente acoplado, encapsulado, asertivo, no redundante.

Bad smell:
Señal de mal diseño de código como duplicidad, ilegible o complicado. No resultan en errores pero futuros desarrollos pueden ser más difíciles o demandar más hrs. de trabajo.
Indicios de problemas q' requieren aplicar de refactoring, siendo indicante de mala calidad de código.

Bloaters: código, métodos y clases que crecen gigantescos y es complicado trabajar c/ellos. Estos bad smells surgen c/ el paso del tiempo de la evolu del programa.

① **Long method** es un método que tiene muchas líneas de código. Se considera q' es largo cuando tiene más de 20 o 30 líneas, depende del lenguaje. Es difícil entenderlo, cambiarlo y reusarlo.
Se aplicaran los refactors de extract method/descompose conditional/replace temp with query/descompose conditional w/ polymorphism.

② **Large class** una clase intenta hacer mucho trabajo, tiene muchas variables de instancia o métodos. Usual' es código duplicado.
Refactors: extract class / extract subclass.

③ **Data clumps** a veces, <> partes del código contienen grupos idénticos de variables. Estos grupos deben convertirse en sus propias clases.
Refactoring: extract class, introduce parameter object, preserve whole object.

④ **Long parameter object list** un mismo método c/ una larga lista de parámetros es + difícil de entender, también es difícil de obtener todos los parámetros p/ pasarlos en la llamada entonces el método es difícil de reusar.
Refactor: replace parameter with method, preserve whole object, introduce parameter object.

⑤ **primitive obsession** usa primitivos en vez de pequeños objetos como listas, arrays, etc. Si tengo gran cantidad de campos primitivos sería lógico agruparlos de alguna forma.
Refactoring: replace data value with object / introduce parameter object / preserve whole object / replace array with object.

Change preventers: indican que nuevos cambios son difíciles de implementar xq requieren modificar múltiples lugares.

① **divergent change:** si un cambio en una clase requiere cambiar varias partes de la clase.
Refactor: extract class

② **shotgun surgery:** si un cambio en una clase requiere cambiar en cascada en varias clases
Refactor: move method/field.

NOTA

Scanned with CamScanner

parallel inheritance ocurre cuando las subclases de la clase A deben estar vinculadas a las subclases de la B
Refactoring: move method/move field.

Couplers hay acoplamiento excesivo of clases reduciendo flexibilidad y particion.

Driver Feature envy: un metodo de una clase usa los datos y métodos de otra clase para hacer su trabajo, este bad smell indica q' el metodo fue ubicado en una clase incorrecta.
Refactoring: move method

Inappropiate intimacy una clase usa campos y metodos de otra clase
Refactoring: move method/field, self encapsulate field

Message chains: ocurre cuando un objeto llama a otro objeto, este a otro y asi sucesivo.
Refactoring: hide delegate, extract/move method

Middle man si tengo una clase q' solo realiza una accion, delegando trabajo a otra.
Refactoring: remove middle man

Tool abusers implementan erroneo o nula de principios de POO.

Switch statements uso de operador switch/if que se vuelve complejo.
Refactoring: replace conditional with polymorphism

Refused bequest una subclase heredado no usa la mayoria de los métodos
Refactoring: push down method/field

Temporary field la clase tiene variables de instancia q' solo se usa en algunos metodos y no siempre se define/inicializa.
Refactoring: extract class, replace method with method object.

Dispensables: código sin sentido y su ausencia haria q' el código fuera más limpio, eficiente y fácil de entender

Lazy class: clase que rara vez se usa y podria ser innecesaria y su funcionalidad podria formar parte de otra clase.
Refactoring: inline class, collapse hierarchy.

Speculative generality hay clases y métodos sin usar q' se tienen x las dudas
Refactoring: collapse hierarchy (p/ abstractas sin usar), inline class (p/ delegar funcionalidades a otras clases), remove parameter (metodos parametros q' no uso en metodos)

Data class: clase q' solo tiene variables y getters/setters. Sucede cuando otras clases tienen métodos d "envidia de atributo", indica q' esos metodos deberian estar en la data class
Refactoring: move method

Duplicate code: mismo código o muy similar, Malo q' el código más largo de lo q' tiene que ser, es dificil de mantener y cambiar.
Refactoring: extract method, pull up method, form template method

God class: una sola clase asume demasiadas responsabilidades, volviéndose excesivo grande, complejo y dificil de mantener.

NOTA

HOJA Nº 5

FECHA

Feature envy vs. inappropiate intimacy.

Feature envy: ocurre cuando un método usa más características de otra clase que las propias y como consecuencia conduce q' el diseño se enrede en dos clases.

Inappropiate intimacy: compromete la encapsulación de la otra clase. Por ejemplo accediendo directamente a las variables de instancia a las q' no se debe acceder directo'

Divergent change vs. shotgun surgery

En divergent change: muchos cambios son hechos en una sola clase

Shotgun surgery: se refiere a cuando un solo cambio genera mucho cambio en otros clases.

Catálogo de refactoring

Permiten distribuir el código adecuadamente

Extract method: es código que puedo agrupar x lo tanto lo muevo a un nuevo método y reemplazo ese código x una llamada a ese método.

```
void printOwing(){
    printBanner();
    System.out.println("name:" +name);
    System.out.println("name:" +algo);
}
```

```
void printOwing(){
    printBanner();
    printData();
}
void printData(){
    System.out.println("name"+name);
    System.out.println("surname"+surname);
}
```

long method, message Chain, duplicate Code

Replace method with query: coloco una exprex en una variable local p/ posterior' usarlo. La solux es mover la exprex a un método separado y devuelvo el resultado

```
double calcularTotal(){
    double basePrice=quantity*itemPrice;
    if (basePrice>1000){
        return basePrice*0.95;
    }else{
        return basePrice*0.98;
    }
}
```

=>

```
double calcularTotal(){
    if(basePrice()>1000){
        return basePrice()*0.95;
    }else{
        return basePrice()*0.98;
    }
}
double basePrice(){
    return quantity*itemPrice;
}
```

long method

Reemplace method with method object: tengo un método largo en la cual las variables locales están entrelazadas. Se transforma el método en una clase separada x/q las variables locales se conviertan en campos de la clase.

```
public double price(){
    double primaryBasePrice;
    double secondaryBase;
    double terciaryPrice;
    ...
}
```

```
public double price(){
    return new PriceCalculator(this).compute();
}
class PriceCalculator{
    double primaryBase;
    double secondaryBase.
    ...
    public double compute(){
    }
```

temporary field

NOTA

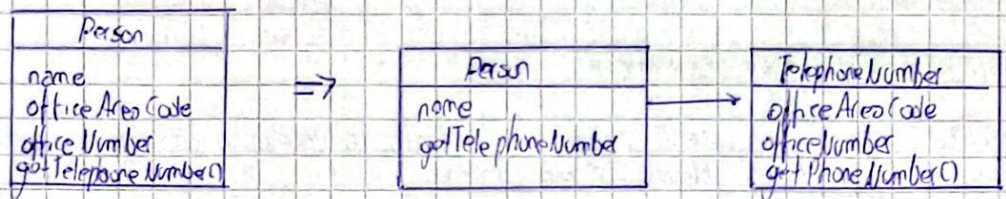
Move method un método es usado más en otra clase q' en su propia clase, esto mal asignada. Un método está usando o usará muchos servicios q' están definidos en una clase <> a la suya.
La solux es mover el método a la clase donde están los servicios que usa y convertir el método original en un simple delegado.

shotgun surgery, feature envy, inappropiate intimacy, alternative class, data class.

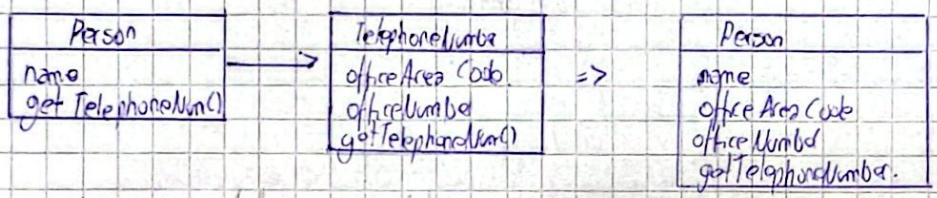
Move field un campo es usado más en otra clase que en la original

shotgun surgery, inappropiate intimacy

Extract class cuando una clase hace el trabajo de dos, solux: crear una nueva clase y acomodar la responsabilidad de los campos y métodos donde corresponda.



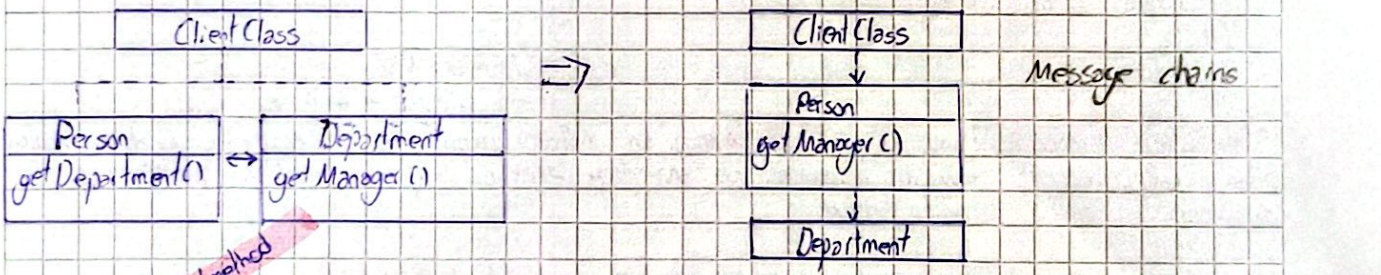
Inline class una clase no hace nada ni tampoco tiene responsabilidades → se mueve todo a otra clase



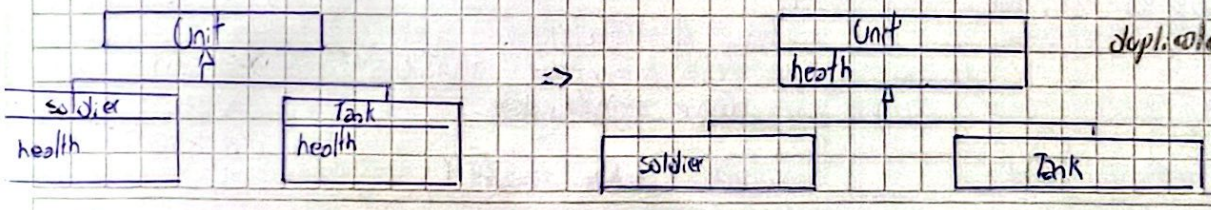
Lazy class, speculative generality

Remove middle man una clase tiene muchos métodos q' simplemente delegan a otros objetos, la solux es eliminar esos métodos y forzar al cliente a llamar a los métodos finales.
middle man

Hide delegate el cliente obtiene el objeto B de un campo o método del objeto A. Luego, el cliente llama a un método del objeto B. La solux es crear un nuevo método en la clase A. Ahora el cliente no conoce ni depende de la B



Pull up field dos clases tienen un mismo campo. La solux es sacar el campo de las subclases y llevarlo al padre



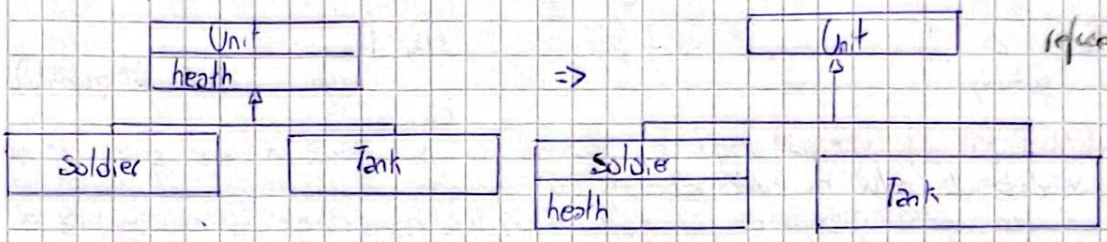
NOTA

Scanned with CamScanner

HOJA Nº 6

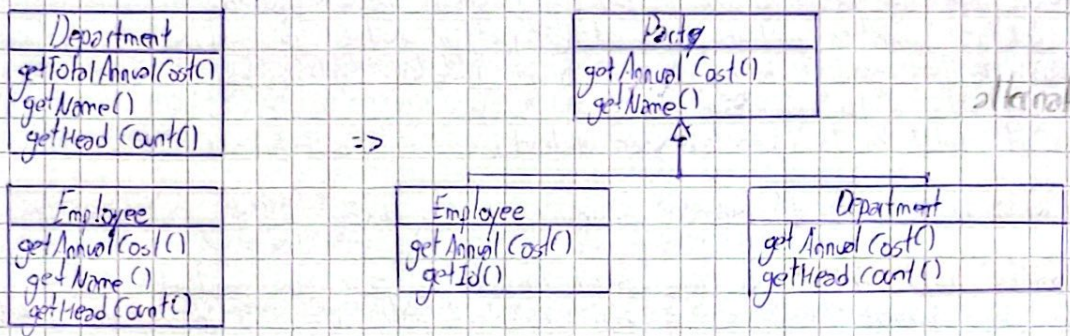
FECHA

push down field / method el campo es usado en pocas subclases entonces el campo se baja a las subclases q lo necesiten



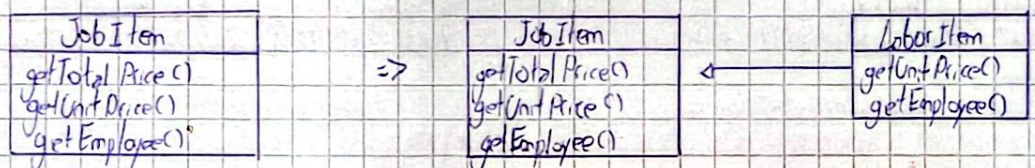
refused bequest

extract superclass hay 2 campos y métodos comunes x lo q se crea una superclase compartida de ellos y mover los campos y métodos en común.



alternative class

Extract subclase una tiene algunas características q se usan pocas veces. x lo q se crea una subclase y se mueven.

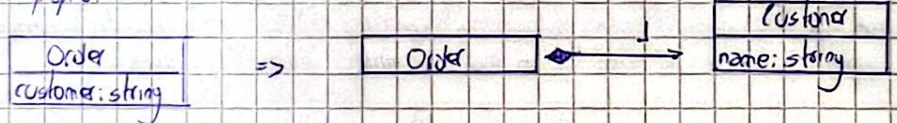


large class

Self encapsulated field se accede a los campos del objeto sin usar getters/setters x lo q se crean estos métodos p/ cada campo.

inappropriate intimacy

Replace data value with object una clase contiene un campo de datos el cual tiene comportamiento propio.



primitive obsession

Replace array with object hay un arreglo que tiene varios tipos de datos x lo q se reemplaza un arreglo c/ un objeto q separa los campos en c/ elemento.

```
String[] row = new String[2];
row[0] = "Liverpool";
row[1] = "15";
```

=>

```
Performance row = new Performance();
row.setName("Liverpool");
row.setWins("15");
```

primitive obsession

NOTA

Scanned with CamScanner

Descompose conditional tengo un conditional complejo c/ if/else x lo q' se descompone en métodos separados.

```
if (date.before(SUMMER_START) || date.after(SUMMER_END)) {
    charge = quantity * winterRate + winterServiceCharge;
} else {
    charge = quantity
}
```

=>

```
if (isSummer(date)) {
    charge = summerCharge(quantity);
} else {
    charge = winterCharge(quantity);
}
```

long method

Replace conditional with polymorphism tengo un condicional q' hace varias acciones s/ el tipo de objetos o propiedades. La solu es crear subclases q' coincidan c/ las ramas del condicional, en ellas crear un método compartido y mover el código correspondiente. Luego, reemplaza el condicional con la llamada al método importante.

① Crear la jerarquía de clases
② Si el condicional es parte de un método largo, aplico extract method
③ x cada subclase creo un método q' sobreescribe al método q' contiene el condicional, copio el código de la condix correspondiente a el método de la subclase, borro condix y código del método en la superclase
④ Hacer q' el método en la superclase sea abstracto.

switch statements

Rename method el nombre del método no explica lo q' hace x lo q' se renombra.

Replace parameter with method se llama a un método y pasar sus resultados como parámetros de otro método mientras q' ese método podría llamar a la consulta directo' x lo q' en vez de pasar el valor a través de un parámetro, colocar una llamada de consulta dentro del cuerpo del método.

```
int basePrice = quantity * itemPrice;
double seasonDiscount = this.getSeasonalDiscount();
double fees = this.getFees();
double finalPrice = discountedPrice(basePrice, seasonDiscount);
```

=>

```
int basePrice = quantity * itemPrice;
double finalPrice = discountedPrice(basePrice);
```

long parameter list.

Preserve whole object tengo varios valores de un objeto luego tengo que pasarlos como parámetros p/ un método x lo q' se pasa el objeto entero.

```
int low = daysTempRange.getLow();
int high = daysTempRange.getHigh();
boolean withinPlan = plan.withinRange(low, high);
```

=>

```
boolean withinPlan = plan.withinPlan(daysTempRange);
```

data clumps, preserve whole object.

Form template method la subclase implementa pasos q' contienen pasos similares en el mismo orden x lo cual se mueve la estructura de pasos a una superclase y dejar la implementax de <> pasos en las subclases x lo cual aplico el template method patron.

Flamewalks (3)

Libreries

Una librería de clases es un conjunto de clases q' puedo usar independiente' pero todas tienen una relax conceptual.

Cada clase en la librería resuelven un problema concreto, independiente al contexto de la aplicación, no espera nada de nuestro código y general' independiente a otras clases de la librería.

NOTA

Frameworks

Aplicación semi-completa y reusable q puede ser especializada p/ producir aplicaciones a medida. La cual db huecos p/ q se rellice.
Las clases en el framework se relacionan de manera q resuelven la mayor parte del problema en cuestión, conformando un todo. Se relacionan mediante herencia, envío de mensaje, hooks, etc.

Las clases trabajan en un todo. Los desarrolladores incorporan el Framework en sus aplicaciones y lo especializan p/ cubrir las necesidades. Donde el framework define el esqueleto y el desarrollador define sus propias características p/ completarlo.
A diferencia de un toolkit/librería, un framework provee una estructura coherente en lugar de un conjunto de clases útiles.
Es el framework quien dice cómo hacer las cosas. El framework controla al nuestro, conociéndose como inversión de control, decidiendo cuándo ocurren las cosas.

Frameworks de infraestructura brindan una infraestructura portable y eficiente sobre el cual construir una variedad de aplicaciones, q tienen en plataformas de desarrollo y facilitan tareas de desarrolladores.

Framework de integrar usados p/ integrar componentes de app distribuidas y aumentan la capacidad p/ modularizar, reusar y extender su infraestructura.

Framework de aplicación frameworks específicos p/ un determinado dominio.

Frozenspots de un framework son aspectos que serán igual p/ todos los artefactos construidos c/ el framework, NO podemos cambiarlos.

Hotspots de un framework partes del artefacto a construir c/ el framework q puedo cambiar.
Son puntos de extensión/ganchos q nos permiten introducir variantes y así construir aplicaciones diferentes. Son las partes del artefacto a construir c/ el framework q puedo cambiar. Algunos puedo y otro debo instanciarlos.
Instanciar un hotspot puede requerir una combinación de acciones como herencia y composición.

Herencia (caja blanca) como desarrollador
- Implementa, extiendo y redefino métodos, las variables y los métodos son heredados.
- Podría cambiar cosas q el desarrollador no tuvo en cuenta.
- Puedo extender el framework
- Debo aprender qué heredo y qué puedo hacer c/ ello.
- No puedo heredar comportamiento de otro lado.

Como desarrollador del framework
- Dejo ganchos en clases del framework
- Paso el control c/ mensajes self
- No necesito pensar todos los hotspots y todos sus casos.
- No necesito pasar estado como parámetros.

NOTA

Composix (Caja negra)

Instancio y configuro
Conecto a mi código el callbacks
Solo conozco algunas clases del framework y sus mensajes.
No puedo cambiar o extender el framework
Solo tengo acceso a los objetos q recibo de los callbacks.

Caja blanca

A los frameworks q usan herencia p/ implementar hotspots se conocen como caja blanca x lo cual se usan variables y métodos heredados. Se pueden extender, implementar y redefinir métodos lo cual llevo a la inversión de control.
Los ganchos están en las clases abstractas y concretas.

Caja negra

Utilizo composix y debo saber poco del código hecho.

Plantillas y ganchos

Son una estrategia de programax comúnmente utilizado p/ introducir puntos de variabilidad (hotspots) en los frameworks orientados a objetos.
Es decir, las plantillas y los ganchos pueden utilizarse p/ separar la funcionalidad constante (frozenspot) de la funcionalidad variable (hotspot) en un framework. Básicamente, las plantillas implementan lo que se mantiene constante, los ganchos lo que varía. Estas pueden implementarse utilizando herencia o composix.

Inversión de control

La inversión de control es una característica clave en la arquitectura del framework. Permite q los pasos fundamentales del procesamiento de una aplicax. q son comunes a todas las instancias, puedan ser personalizados x objetos llamados manejadores de eventos. Estos manejadores son invocados x el framework de manera automática, como parte de su mecanismo de despacho.

Hotspots c/ herencia

El template method asegura q se cumplan los pasos.
Programamos x diferencia implementando los hooks, utilizando herencia en donde la plantilla se implementa en una clase abstracta y los ganchos en sus subclases.

Hotspots c/ composix

El template method asegura q se cumplan los pasos.
Implementamos los hooks, usando la composix en donde un objeto implementa la plantilla y delega la implementax de los ganchos en sus partes.
No tenemos acceso a las variables de instancia ni métodos privados ni de otras componentes.

NOTA

Scanned with CamScanner

HOJA Nº 0

FECHA

④ Testing

Test de unidad es el testeo de la mínima unidad de ejecuc. donde en POO, es la un método.
El objetivo es aislar cada parte del programa y mostrar que funciona correcto'. Cada test confirma q' un método produce el output esperado ante un input conocido.
Hay <> estrategias
Particiones equivalentes: considera el dominio de la funcionalidad a testear, si paso el test, otros del mismo conjunto pasarán.
Valores borde: se testean escenarios d/ valores en los límites de particiones.

Test double crea un objeto q' es una maqueta (polimórfica) del objeto o módulo requerido y se usa la maqueta s/ se necesita. Se generan <> patrones q' se aplican a cada caso.

Test STUB es un objeto usado en pruebas unitarias q' proporciona respuestas predefinidas a llamadas realizadas x el código bajo prueba. Los stubs se configuran p/ simular el comportamiento de las dependencias reales y pueden devolver valores específicos o generar excepciones controladas. Los stubs se usan p/ aislar el código bajo pruebas y verificar cómo responde a diferentes resultados

Test spy test stub + registro de outputs. Es un objeto usado en pruebas unitarias q' registra información sobre las llamadas realizadas x el código bajo prueba a sus dependencias. A diferencia de los stubs, los spies no definen expectativas de llamadas y no generan errores si las llamadas esperadas se ocurren. En lugar de eso, se utilizan p/ verificar el comportamiento posterior' inspeccionando las llamadas registradas. Los spies son útiles p/ verificar qué llamadas registradas se realizaron y con qué parámetros, permitiendo analizar el comportamiento del código bajo prueba.

Mock object test stub + verification of outputs. Es un objeto utilizado en pruebas unitarias q' se configura c/ expectativas de llamadas específicas. Los mock objects se utilizan p/ verificar las interacciones d/ el código bajo prueba y sus dependencias. Se definen expectativas de llamadas y se comprueba si se han cumplido durante la prueba. Si alguna expectativa no se cumple, se genera un error de prueba.
Son más rigurosos q' los stubs y se centran en verificar las interacciones correctas d/ objetos.

Test Driven Development
Combina test first development donde se escriben el test antes del código q' haga pasar el test. y el refactoring.
Se piensa en el diseño y q' se espera de cada requerimiento antes de escribir código q' sea limpio y funcione.

NOTA

* replace temp with query: tengo una variable q/ una expr q/ puede ser resultado. entonces lo muevo a un método.
* extract subclass = features solo usadas en algunas hijas = lo bajo, idem pull up down field/method.
* introduce parameter object: agrupo variables en un único objeto.
* preserve whole object: pasar todo el objeto en vez de parámetro < parámetro.
* extract method: mueve código a un método.

BAD SMELLS

BLOATERS	código, métodos y clases que podrían crecer rápidamente
CHANGE PREVENTERS	cambio que conlleva cambios en otros lados
COUPLERS	contribuye al alto acoplamiento
TOOL ABUSERS	abusa o usa mal los principios de la POO
DISPENSABLES	algo sin sentido y la ausencia haría un código más limpio
BAD SMELL Bloaters	**REFACTORING QUE LO CORRIGE**
long method	extract method, despompose conditional, replace temp with query,
large class	extract class, extract subclass*
variables repetidas en <> partes data clumps	extract class, introduce parameter object, preserve whole object
long parameter list	replace parameter with method, preserve whole object, introduce parameter object
uso en lugar de variables primitive obsession	replace data value with object, introduce parameter object, replace array with object
Change preventers	
divergent change	extract class
combo q/ requiere más cambios. shorgun surgery	move method/ field
Couplers	
feature envy	move method
inappropiate intimacy	move method/ field, self encapsulate field
message chains	hide delegate, extract method, move method
middle man	remove middle man
Tool abusers	
switch statements	replace condicional with polymorphism
no usa métodos/atributos heredados refused bequest	push down method/ field
hacen lo mismo pero c/ <> nombres alternative class	rename method
temporary field	extract class, replace method with method object
Dispensables	
lazy class clase poco usada	inline class, collapse hierarchy
speculative generality	collapse hierarchy, inline class, remove parameter
data class	move method
duplicate code	extract method, pull up method, form template method
comments	extract method, rename method

mueve todo a una única clase

mueve subclase a superclase.

capas/métodos < las clases

mueve código duplicado a un único método

mueve código duplicado de hijos al padre.

Scanned with CamScanner